

EDUGENIE – GOOGLE GEMINI POWERED LEARNING ASSISTANT

REQUIREMENTS DOCUMENT

1. Project Overview

Project Name: EduGenie – Google Gemini Powered Learning Assistant

Project Type: Generative AI Based Educational Web Application

Primary Technology: Python, FastAPI, Google Gemini API

Frontend: HTML, CSS, Jinja2 Templates

AI Technology: Google Gemini Generative AI

Optional Local AI Model: LaMini-Flan-T5-783M

EduGenie is a lightweight AI-powered learning assistant designed to help students understand educational content more effectively. The system provides question answering, concept explanation, quiz generation, text summarization, and personalized learning recommendations through a simple web interface.

The application uses a FastAPI backend to process user requests and connects the appropriate learning operation to a Generative AI model. The current implementation uses a configurable Gemini model, with gemini-2.5-flash as the default model, while a local LaMini-based explanation model can optionally be enabled.

2. Problem Statement

Students often spend considerable time searching for explanations, preparing practice questions, summarizing study material, and deciding what topics to study next.

Traditional learning resources may not provide immediate, personalized assistance. Students therefore require a system that can:

- Answer academic questions quickly.
- Explain difficult concepts at an appropriate level.
- Generate practice questions automatically.
- Summarize lengthy educational content.
- Suggest suitable learning topics and a learning sequence.

EduGenie addresses these requirements by integrating Generative AI capabilities into a simple educational web application.

3. Purpose of the System

The main purpose of EduGenie is to provide an accessible AI-powered learning assistant that supports students during self-learning and revision.

The system should transform user-provided educational questions or content into useful learning outputs such as:

- Direct answers.
 - Simplified explanations.
 - Multiple-choice quizzes.
 - Concise summaries.
 - Personalized learning recommendations.
-

4. Scope of the Project

The scope of the current system includes:

1. AI-based question answering.
2. Concept explanation.
3. Automatic quiz generation.
4. Text summarization.
5. Personalized learning-path recommendations.
6. Web-based interaction through a browser.
7. REST API endpoints using FastAPI.
8. Gemini API integration.
9. Optional local explanation model support.
10. Automated testing using mocked Gemini responses.

The project is intentionally lightweight and does not currently include user authentication, persistent student accounts, progress databases, or production-scale infrastructure.

5. Stakeholders

The main stakeholders are:

5.1 Students

Students are the primary users of the application. They can use EduGenie for understanding concepts, revision, practice, and learning recommendations.

5.2 Teachers and Educators

Teachers can use the system as an assisting tool for preparing practice questions, explanations, and learning material.

5.3 Project Developers

Developers are responsible for implementing, testing, maintaining, and extending the application.

5.4 Educational Institutions

The system can potentially be used as a supplementary AI-based learning tool in educational environments.

6. Functional Requirements

FR-01: User Question Answering

The system shall allow the user to enter an academic question.

The system shall:

1. Receive the question from the frontend.
2. Send the question to the appropriate backend endpoint.
3. Process the request using Gemini.
4. Generate an AI-based response.
5. Return the answer to the user.

Endpoint: /qa

Example:

User Question: Which is the largest ocean?

Expected system behavior:

Generate a concise educational answer explaining the largest ocean.

7. Concept Explanation Requirement

FR-02: Concept Explanation

The system shall allow users to request an explanation of a particular concept.

The user should be able to provide:

- Topic or concept.
- Desired learning level, such as beginner.

The system shall generate an explanation appropriate to the requested level.

Endpoint: /explain

Example:

Topic: Pythagoras Theorem

Level: Beginner

The system should provide an understandable explanation rather than only a short definition.

The current project supports an optional local LaMini explanation model. When local explanation is disabled or unavailable, the application can use Gemini as a fallback.

8. Quiz Generation Requirement

FR-03: Automatic Quiz Generation

The system shall generate a quiz from educational content supplied by the user.

The quiz functionality shall:

1. Accept a passage or topic.
2. Send the content to the AI model.
3. Generate three multiple-choice questions.
4. Provide structured question data.
5. Include answer information in the generated result.

Endpoint: /quiz

The system should produce quiz content that is relevant to the supplied educational material.

The current implementation uses Gemini structured output for quiz generation.

9. Text Summarization Requirement

FR-04: Text Summarization

The system shall allow users to provide a passage or educational text.

The system shall:

1. Accept the input passage.
2. Process the passage using Gemini.
3. Identify the important information.
4. Produce a concise summary.
5. Return the summary to the user.

Endpoint: /summarize

The summary should preserve the major ideas while reducing unnecessary detail.

10. Personalized Learning Recommendation Requirement

FR-05: Learning Recommendations

The system shall provide learning recommendations based on:

- Subject or topic.
- User's current learning level.

The system shall use Gemini to generate a recommended learning sequence.

Endpoint: /learn/recommendations

Example:

Subject: SQL

Level: Beginner

Expected output:

A structured sequence of topics that a beginner should study to develop knowledge of SQL.

11. Frontend Requirements

FR-06: Web Interface

The system shall provide a browser-based interface.

The frontend shall allow users to:

- Select the desired learning operation.
- Enter a question, topic, or passage.
- Submit the request.
- View the AI-generated result.

The frontend is implemented using HTML/Jinja2 templates with CSS styling.

12. Backend Requirements

FR-07: FastAPI Backend

The backend shall be implemented using FastAPI.

The backend shall:

- Receive HTTP requests.
- Validate request data.
- Route requests to the appropriate functionality.
- Communicate with Gemini.
- Return generated responses.
- Provide API documentation.

The application exposes interactive API documentation through the /docs endpoint.

13. Generative AI Requirements

FR-08: Gemini Integration

The system shall integrate with Google's Gemini Generative AI service.

The Gemini model shall be configurable through environment/configuration settings.

The current implementation uses gemini-2.5-flash as the default configurable model.

Gemini is used for functions including:

- Question answering.
- Concept explanation fallback.
- Quiz generation.
- Summarization.

- Learning recommendations.

14. Local AI Model Requirement

FR-09: Optional Local Explanation Model

The system may optionally support a local LaMini-Flan-T5-783M model for concept explanation.

When enabled:

- The local model processes explanation requests.
- The model is downloaded when required.
- The application can operate without using Gemini for that explanation operation.

The local model is optional because CPU execution can be slower and more memory intensive. The default configuration keeps local explanation disabled.

15. API Requirements

The application shall provide the following main API endpoints:

Endpoint	Purpose
/qa	Question answering
/explain	Concept explanation
/quiz	Quiz generation
/summarize	Text summarization
/learn/recommendations	Learning recommendations
/health	Application health check
/docs	FastAPI interactive API documentation

These endpoints form the main functional interface between the frontend and backend.

16. Input Requirements

The system shall accept text-based educational inputs.

Examples include:

Question Input

Which is the largest ocean?

Explanation Input

Topic: Pythagoras theorem

Level: Beginner

Quiz Input

Water cycle and its stages

Summary Input

Photosynthesis is the process by which green plants...

Learning Recommendation Input

Subject: SQL

Level: Beginner

17. Output Requirements

The system shall return AI-generated educational content.

Depending on the selected operation, the output may be:

- Natural-language answer.
- Concept explanation.
- Structured quiz.
- Summary.
- Learning roadmap/recommendations.

Quiz responses should follow the structured format expected by the application.

18. Non-Functional Requirements

NFR-01: Usability

The application should have a simple interface that can be used by students without advanced technical knowledge.

NFR-02: Performance

The system should process normal educational requests within a reasonable response time, subject to Gemini API response time and network conditions.

NFR-03: Reliability

The application should handle invalid or failed requests without crashing the entire application.

AI service failures should be handled appropriately.

NFR-04: Maintainability

The application should use a modular structure so that individual learning functions can be modified or extended independently.

NFR-05: Scalability

The architecture should allow future additions such as:

- Authentication.
 - Student profiles.
 - Learning progress tracking.
 - Persistent storage.
 - Additional AI models.
 - Additional educational tools.
-

NFR-06: Security

The Gemini API key shall not be hard-coded into the source code.

The API key shall be stored using environment configuration.

The .env file should not be committed to the Git repository.

NFR-07: Compatibility

The application should run on systems supporting Python 3.10 or later.

It should be accessible through a modern web browser.

NFR-08: Testability

The system shall provide automated tests for its main functionality.

Tests should be capable of running without making paid Gemini API requests by mocking Gemini responses.

The project documentation specifies running the test suite using:

pytest -q

19. Hardware Requirements

The basic version of EduGenie does not require specialized hardware.

Minimum Requirements

- Modern computer or laptop.
- Minimum 4 GB RAM recommended.
- Internet connection for Gemini API requests.
- Standard processor capable of running Python applications.

For Optional Local AI Model

Additional RAM and processing capability may be required because the local LaMini model can be resource-intensive, particularly when running on CPU.

20. Software Requirements

Operating System

The application can be developed and executed on:

- Windows
- Linux
- macOS

Programming Language

- Python 3.10+

Development Environment

- Visual Studio Code or another Python-compatible IDE.

Backend Framework

- FastAPI

Server

- Uvicorn

Frontend Technologies

- HTML
- CSS
- Jinja2

AI Service

- Google Gemini API

Optional AI Model

- LaMini-Flan-T5-783M

Testing

- Pytest

21. Environment Configuration Requirements

The application requires a Gemini API key.

The configuration should contain:

GEMINI_API_KEY=your_api_key_here

The actual API key must remain private and must not be uploaded to GitHub.

Optional local-model configuration can be enabled through the application's configuration mechanism.

22. Dependency Requirements

The project requires Python packages for:

- FastAPI

- Uvicorn
- Gemini/Google GenAI integration
- Jinja2
- Environment configuration
- Testing

Additional dependencies may be installed when the optional local explanation model is enabled.

The project separates the optional local-model dependencies from the default application requirements.

23. Deployment Requirements

For local development, the application shall be started using:

```
uvicorn main:app --reload
```

The application can then be accessed through:

```
http://127.0.0.1:8000
```

FastAPI API documentation is available through:

```
http://127.0.0.1:800
```